

Lab5 part2: Tiger Compiler without register allocation

ddl 2026/6/10 23:59

1. Lab Description

In lab5-2, you need to translate the Tree Language into assembly codes. If you finish this lab, you will get a complete runnable tiger compiler. You need to make sure that your compiler generate working code that runs on x86-64 platform, you can skip register allocation in this lab.

Related files for this lab are:

- `src/tiger/frame/*.*` Files related to function stack frame(chapter 6)
- `src/tiger/translate/*.*` Files related to IR tree translation(chapter 7)
- `src/tiger/canon/*.*` Files related to basic blocks & traces(chapter 8, this part has already been implemented)
- `src/tiger/codegen/*.*` Files related to assembly code generation(chapter 9)
- `src/tiger/runtime/runtime.c` Tiger program runtime file(will be linked with code files generated by your compiler)
- `src/tiger/env/env.*` The EnvEntry classes help the compiler store the information of variables and functions
- `src/tiger/output/output.*` Files related to output assembly for frags.

To finish this lab, you will only need to finish the following modules: { **x64 stack frame** } { **IR tree translation** } { **code generation** } you can modify any file included in the `make ziplab5-2`:

Code block

```
1  .— src/tiger/parse/tiger.y
2  |— src/tiger/lex/tiger.lex
3  |— src/tiger/lex/scanner.h
4  |— src/tiger/absyn/absyn.*
5  |— src/tiger/semant/semant.*
6  |— src/tiger/escape/escape.*
7  |— src/tiger/frame/frame.h
8  |— src/tiger/frame/temp.*
9  |— src/tiger/frame/x64frame.*
10 |— src/tiger/translate/translate.*
```

```
11 |— src/tiger/translate/tree.*
12 |— src/tiger/canon/canon.*
13 |— src/tiger/codegen/assem.*
14 |— src/tiger/codegen/codegen.*
```

 **Notice:** Before you start this lab, you should carefully read chapters 6, 7, 8, 9, 12 of the textbook.

Important Notes

1. If you have not finished your translation in lab5-part1 yet, **complete codes for IR tree translation in this lab first.**
2. Before implementing IR tree translation, carefully read chapters 6 and 7 of the textbook.
3. Before you start implementing code generation, carefully read the calling convention of the x86-64 platform, you can reference [OSDEV calling conventions cheat sheets](#) or [System V ABI AMD64 arch P123~P124](#) for details.
4. The file `runtime.c` is a C-language file containing several external functions useful to your Tiger program. These are generally reached by `externalCall` from code generated by your compiler.
5. Carefully read chapter 12, which gives some useful interface declarations you may use in the module ***frame***.
6. You can only use one register(`%rsp`) as the stack pointer. `%rbp` is not allowed to be used as the stack pointer.
7. After you finish the module ***codegen***, remember to check `output.cc` for the output of your assembly code, we have annotated some code for passing compilation.
8. You can read `src/tiger/main/test_codegen.cc` to figure out the order in which modules are called and the workflow of the tiger compiler.

 This lab is still extremely difficult and hard to debug. **Please leave yourself enough time(at least two weeks) to finish this lab.**

Tiger Interpreter

Since you have not finished the register allocation part yet, your assembly can't link using GCC. We use Tiger Interpreter to help you judge if your answer is right. This interpreter takes assembly file(.s) as input and interprets your assembly code even without register allocation. Your final assembly code should act as follows:

Code block

```
1  movq t112, %rsp
2  addq %rcx, %rbx
3  leaq L1(t118), %rdi
```

This code doesn't make any sense, but it shows what assembly code you should generate in this lab, where `t1xx` represents a temporary register not allocated yet. Note that you can use machine registers now, and you must use them for some special instructions (like `imul/idiv`, `ret` value). We only support the following instructions now: **`movq`, `addq`, `subq`, `imulq`, `idivq`, `leaq`, `callq`, `cmpq`, `jmp`, `je`, `jne`, `jg`, `jge`, `jl`, `jle`, `retq`, `cqto`**, which is enough for you to finish this lab.

Besides, we have provided a debugger inside this interpreter to help you figure out what is going wrong, details can be found in the following Appendix A. Tiger Interpreter's source code is in `scripts/lab5_test`, and is written in python.

Implementation Tips

1. As discussed in class, when performing canonicalization, expressions that contain side effects —such as `CALL(f())`—must be evaluated before the surrounding expression that uses them. This means extracting the side-effecting expression, assigning its result to a temporary variable, and replacing the original sub-expression with that temporary in the pure expression. This transformation ensures that all expressions become side-effect-free, which simplifies code generation and allows for more effective optimizations. For implementation details, refer to the files under `src/tiger/canon/`.
2. Figure out what information a frame typically stores. Additionally, you must distinguish between **caller-saved** and **callee-saved** registers:
 - *Caller-saved registers* must be saved by the calling function if it wants to preserve their values across a function call.
 - *Callee-saved registers* must be saved and restored by the callee function if it uses them.
3. In the x86-64 System V ABI: The first six function arguments are passed via registers and the remaining arguments go on the stack. Access the rest of them using offsets from a saved frame pointer.
4. Don't forget to generate proper prologue and epilogue code for each function:
 - In the **prologue**, allocate space on the stack (based on total frame size) and save callee-saved registers if they are used.
 - In the **epilogue**, restore callee-saved registers and deallocate the stack frame before returning.
 - Also make sure to correctly maintain the return address and return value in `%rax`.

2. Environment

Merge the code from lab5-1, and update the branch lab5-2 from gitee.

3. Local Grade Test

The lab environment contains a grading script named **grade.sh**. You can use it to evaluate your code, and that's how we grade your code, too. If you pass all the tests, the script will print a successful hint, otherwise, it will output some error messages. You can execute the script with the following commands.

Code block

```
1  ~/tiger-lab5_2 > make gradelab5-2
2  bash scripts/grade.sh lab5-part2
3  ===== Lab5 Test =====
4  Pass bsearch
5  Pass dec2bin
6  Pass merge/test1
7  Pass merge/test2
8  Pass merge/test3
9  Pass merge/test4
10 Pass prime
11 Pass qsort
12 Pass queens
13 Pass tbi
14 Pass test_array
15 Pass test_patch
16 Pass tfact
17 Pass tfo
18 Pass tif
19 Pass tiffn
20 Pass tlink
21 Pass trec
22 Pass tree
23 Pass twhi
24 [^_^]: Pass
25 LAB5 part2 SCORE: 100
```

4. Submission Instructions

1. Use `make ziplab5-2` command to generate the zip file. Do not change any files other than those included in the `make ziplab5-2` command.
2. Hand in the zip file to Gradescope website.

Appendix: Debug in Tiger

Main Tools

- gdb
- Tiger Interpreter

Instruction Guide

Bug in `test_codegen`

1. Execute `make build-debug`
2. Navigate to the build directory: `cd build`
3. Run (replace `testfile` with the specific testcase to debug, such as `bsearch`)

Code block

```
1  gdb --args ./test_codegen ../testdata/lab5or6/testcases/testfile.tig
```

4. Debug using gdb

Bug in Assembly Code

1. Generate the assembly code:

Code block

```
1  ./build/test_codegen ../testdata/lab5or6/testcases/testfile.tig
```

2. Navigate to the test script directory: `cd scripts/lab5_test`
3. Run

Code block

```
1  python3 main.py -d ../../testdata/lab5or6/testcases/testfile.tig.s
```

4. Debug using the Tiger Interpreter

Tiger Interpreter Supported Commands

- `ni` : Execute the next instruction
- `continue` : Continue execution until the program crashes or hits a breakpoint (similar to `c` in gdb, but also includes the function of `r`)
- `b` : Set a breakpoint in two ways:
 - By line number:
 - Note that the line number corresponds to the compiler' s line numbers, which can be viewed using the interpreter.
 - For example, `b 40` sets a breakpoint at line 40.
 - By function name:
 - This is simpler, but note that external call functions do not follow this process.
 - For example, `b bsearch` sets a breakpoint at the `bsearch` function.
- `disable` : Similar to gdb, but without an `enable` option, making it more like `delete` . Note that the input is the breakpoint number, which can be viewed with `show b` .
- `disassemble` : Similar to gdb, displays the current executing assembly instructions and surrounding instructions. Note that only the line numbers here are accurate.
- `show` : This command includes functionalities of gdb' s `p` , `x` , and `show` :
 - `show registers` : Displays all register values, including mechanical and temporary registers.
 - `show machine` : Displays all mechanical register values.
 - `show temp` : Displays all temporary register values.
 - `show memory` : Outputs current memory address values in order.
 - `show memtable` : Outputs current memory address values in a table format.
 - `show b` : Displays information on all breakpoints.
- `exit` : Exit debug mode.

The functionality is quite limited. If you find any bugs in our source code or have better suggestions, feel free to contact the TA.

Tips

- The Tiger Interpreter's debug mode will halt and perform a disassemble upon encountering any error. You can now use `show` to inspect the problematic registers or memory addresses and check all instructions that modified the relevant addresses or registers, setting breakpoints as needed to identify the issue.

- GDB is a powerful tool, and the following commands are highly recommended:
 - `p` : Very useful for printing function outputs and the values of various object members, as well as performing type casts.
 - `x` : Mainly used to inspect memory addresses.
 - `watch` : Monitors changes to memory addresses.
- Those interested might want to explore `record & replay` , a GDB feature with reverse execution capabilities. It is very useful but has a high learning curve, so choose based on your needs.
- The key to debugging is not the tool but the approach. For a highly modular project like Tiger-Compiler, it is crucial to first locate the error. Especially when there are no exceptions but the results are incorrect, you must learn to pinpoint the error step by step. If necessary, you can read the trace output to determine whether the issue lies in Translation or Codegen.